

FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y
COMPUTACIÓN



DESARROLLO DE APLICACIONES I

Programación Orientada a Objetos

INTEGRANTES:

- QUISPE TICA Ruth Nayeli

DOCENTE:

- FERNANDEZ BEJARANO, Raul Enrique

CICLO: IV

SECCIÓN: A1

FECHA: 28 de septiembre del 2026

HUANCAYO - PERÚ

2026

1. INTRODUCCIÓN

La presente actividad tiene como finalidad aplicar los fundamentos de la Programación Orientada a Objetos (POO) mediante el desarrollo de una aplicación web utilizando el lenguaje de programación Java.

El proyecto denominado ActividadPOO03 ha sido desarrollado utilizando Java 17, Maven, Jakarta Servlet, JSP y Apache Tomcat, permitiendo integrar la programación orientada a objetos con el desarrollo de aplicaciones web.

La aplicación contiene dos problemas prácticos. El primero corresponde a una compra de productos por docenas, donde se determina el importe de la compra, el descuento correspondiente, el importe final a pagar y la cantidad de lapiceros que recibe el cliente como obsequio. El segundo corresponde a la compra de pasajes para la ruta Lima-Huancayo, donde se calcula el importe de la compra, el descuento y la cantidad de caramelos que recibe el cliente.

Para desarrollar estas soluciones se ha utilizado una estructura organizada en modelo, controlador y vista. El modelo contiene las clases que representan los problemas y realizan los cálculos; los Servlets actúan como controladores, recibiendo los datos de los formularios y enviándolos al modelo; finalmente, las páginas JSP permiten la interacción con el usuario y la presentación de los resultados.

De esta manera, la actividad permite comprender que la POO no consiste únicamente en crear clases, sino en organizar la solución de un problema mediante objetos que contienen datos y comportamientos relacionados.

2. OBJETIVOS

2.1 Objetivo general

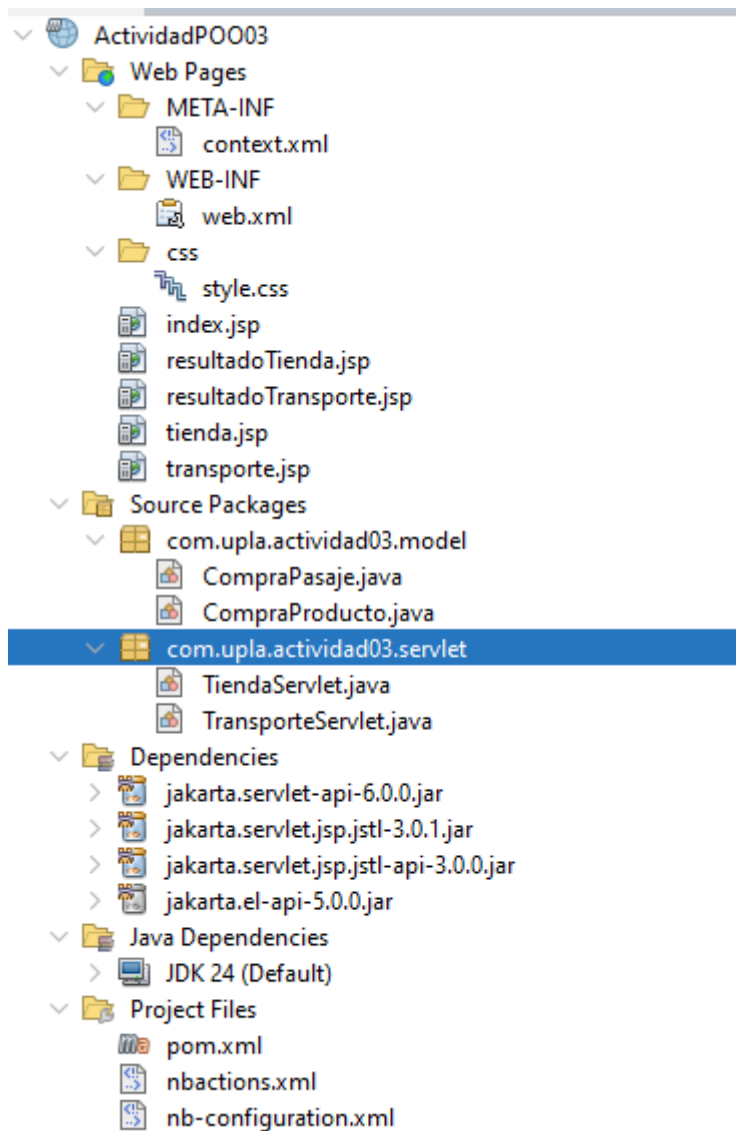
Desarrollar una aplicación web utilizando Java y Programación Orientada a Objetos para solucionar dos problemas relacionados con el cálculo de compras, aplicando clases, objetos, atributos, constructores, métodos, encapsulamiento y validación de datos.

2.2 Objetivos específicos

- Aplicar los conceptos fundamentales de la Programación Orientada a Objetos.
- Crear clases que representen los problemas planteados.
- Utilizar atributos para almacenar los datos necesarios.
- Implementar constructores para inicializar los objetos.
- Crear métodos independientes para realizar cada cálculo.
- Aplicar encapsulamiento mediante atributos privados y métodos públicos.
- Utilizar Servlets para procesar los datos enviados desde los formularios.
- Utilizar JSP para construir las interfaces de usuario.
- Validar los datos ingresados antes de realizar los cálculos.
- Utilizar Maven para administrar la estructura y dependencias del proyecto.
- Ejecutar la aplicación mediante Apache Tomcat.

3. ESTRUCTURA DEL PROYECTO

La estructura principal es:



4. Modelo: **CompraProducto**

La clase **CompraProducto** pertenece al paquete **model**.

Su función es representar y procesar una compra de productos.

Una parte importante del código es:

```
private double precioDocena;  
private int cantidadDocenas;  
private double importeCompra;  
private double importeDescuento;  
private double importeAPagar;  
private int cantidadLapiceros;
```

Estos atributos están declarados como **private**.

¿Por qué se utiliza **private**?

Porque se está aplicando el concepto de encapsulamiento.

El encapsulamiento consiste en proteger los datos internos de una clase y controlar su acceso mediante métodos, principalmente mediante constructores y getters.

Esto evita que otras partes del programa modifiquen directamente los atributos sin control.

5. Constructor de **CompraProducto**

El constructor recibe los datos necesarios para realizar la compra:

```
public CompraProducto(double precioDocena, int cantidadDocenas) {  
    this.precioDocena = precioDocena;  
    this.cantidadDocenas = cantidadDocenas;  
    procesarCompra();  
}
```

El constructor permite crear un objeto **CompraProducto** con los datos ingresados por el usuario.

Por ejemplo:

```
CompraProducto compra =  
    new CompraProducto(25.00, 12);
```

Aquí se crea un objeto que representa una compra de 12 docenas, con un precio de S/ 25.00 por docena.

Lo importante es que después de crear el objeto se ejecuta:

```
procesarCompra();
```

Esto permite realizar automáticamente todos los cálculos.

6. ¿Cómo se calcula la compra?

El método:

```
private void calcularImporteCompra() {  
    this.importeCompra =  
        precioDocena * cantidadDocenas;  
}
```

multiplica el precio de una docena por la cantidad comprada.

Ejemplo:

Si:

Precio = S/ 25

Cantidad = 12 docenas

Entonces:

Importe de compra = 25×12

Importe de compra = S/ 300

Esto demuestra que la lógica de cálculo se encuentra dentro de la clase que representa la compra.

7. ¿Cómo se calcula el descuento?

El programa utiliza dos porcentajes dependiendo de la cantidad comprada:

```
double porcentaje =  
    (cantidadDocenas >= LIMITE_DOCENAS)  
    ? PORCENTAJE_DESCUENTO_ALTO  
    : PORCENTAJE_DESCUENTO_BAJO;
```

El límite establecido es:

```
private static final int LIMITE_DOCENAS = 10;
```

Por lo tanto:

- Si compra 10 o más docenas, obtiene 20 % de descuento.
- Si compra menos de 10 docenas, obtiene 10 % de descuento.

Esto se realiza mediante el operador ternario:

condición ? valorSiCumple : valorSiNoCumple

Por eso el código permite tomar una decisión de forma compacta.

8. ¿Cómo se obtiene el importe a pagar?

El método:

```
private void calcularImporteAPagar() {  
    this.importeAPagar =  
        importeCompra - importeDescuento;  
}
```

Resta el descuento al importe original.

Por ejemplo:

Importe de compra = S/ 300

Descuento = 20 %

Descuento = 300×0.20

Descuento = S/ 60

Importe a pagar = $300 - 60$

Importe a pagar = S/ 240

De esta manera, el programa obtiene automáticamente el monto final que debe pagar el cliente.

9. ¿Cómo se calculan los lapiceros?

El sistema también entrega lapiceros como obsequio cuando se cumple una condición.

```
this.cantidadLapiceros =  
    (importeAPagar >= LIMITE_IMPORTE_OBSEQUIO)  
    ? cantidadDocenas * LAPICEROS_POR_DOCENA  
    : 0;
```

El límite es:

```
private static final double LIMITE_IMPORTE_OBSEQUIO = 200.0;
```

Por lo tanto, si el importe a pagar es igual o superior a S/ 200, se entregan:

2 lapiceros por cada docena

Esto demuestra que el programa no solamente realiza operaciones matemáticas, sino que también implementa reglas de negocio.

10. Modelo: CompraPasaje

La segunda actividad utiliza la clase:

CompraPasaje

Esta clase representa la compra de pasajes de transporte.

Tiene como precio establecido:

```
public static final double PRECIO_PASAJE = 37.5;
```

Esto significa que cada pasaje tiene un valor de:

S/ 37.50

La clase también almacena:

```
private String turno;  
private int cantidadPasajes;  
private double importeCompra;  
private double importeDescuento;  
private double importeAPagar;  
private int cantidadCaramelos;
```

Nuevamente se aplica el encapsulamiento, porque los atributos son privados.

11. Cálculo de los pasajes

El importe de compra se obtiene mediante:

```
importeCompra =  
    PRECIO_PASAJE * cantidadPasajes;
```

Por ejemplo:

Precio del pasaje = S/ 37.50

Cantidad = 10

$37.50 \times 10 = \text{S/ } 375.00$

El programa calcula automáticamente el importe correspondiente.

12. Descuento de los pasajes

Para los pasajes se aplica una regla similar.

15 pasajes o más $\rightarrow$ 8 % de descuento

Menos de 15 $\rightarrow$ 5 % de descuento

La condición se controla mediante:

```
cantidadPasajes >= LIMITE_PASAJES
```

donde:

```
LIMITE_PASAJES = 15;
```

Esto demuestra nuevamente el uso de estructuras de decisión para implementar las reglas del problema.

13. ¿Qué función cumplen los Servlets?

Los Servlets funcionan como intermediarios entre las páginas JSP y las clases Java.

En el proyecto tenemos:

TiendaServlet

TransporteServlet

Su función principal es:

1. Recibir los datos enviados desde el formulario.
2. Convertir los datos al tipo correspondiente.
3. Validar la información.
4. Crear el objeto correspondiente.
5. Ejecutar el procesamiento.
6. Enviar el resultado a una página JSP.

Por ejemplo:

```
String precio =
```

```
request.getParameter("precioDocena");
```

El Servlet obtiene el dato enviado por el formulario.

Después lo convierte:

```
double precioDocena =
```

```
Double.parseDouble(precio);
```

Finalmente crea el objeto:

```
CompraProducto compra =
```

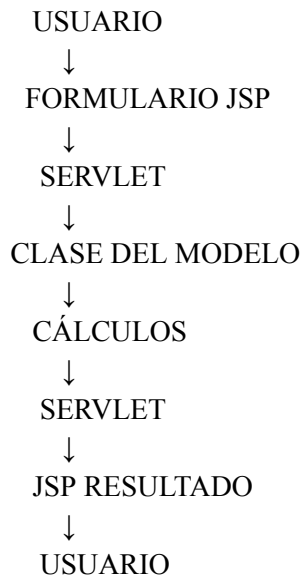
```
new CompraProducto(precioDocena, cantidadDocenas);
```

De esta manera, el Servlet no realiza directamente toda la lógica del negocio, sino que delega el procesamiento a la clase correspondiente.

14. ¿Por qué separar Servlet y modelo?

Esta separación es importante porque permite mantener el código organizado.

Podemos representar el funcionamiento así:



Esta estructura permite separar responsabilidades.

JSP

Se encarga principalmente de la interfaz y presentación.

Servlet

Se encarga de recibir y controlar las solicitudes.

Clase Java

Se encarga de realizar la lógica y los cálculos del negocio.

Esta separación hace que el sistema sea más ordenado y facilita su mantenimiento.

15. Conceptos de POO aplicados

El proyecto permite demostrar varios conceptos importantes de Programación Orientada a Objetos.

Clase

Por ejemplo:

```
public class CompraProducto
```

La clase funciona como una plantilla para crear objetos.

Objeto

```
CompraProducto compra =
```

```
    new CompraProducto(25, 12);
```

Aquí se crea un objeto a partir de la clase.

Atributos

```
private double precioDocena;
```

```
private int cantidadDocenas;
```

Representan las características del objeto.

Métodos

```
calcularImporteCompra();
```

```
calcularImporteDescuento();
```

```
calcularImporteAPagar();
```

Representan las acciones que puede realizar el objeto.

Encapsulamiento

Se aplica mediante atributos privados:

```
private
```

y métodos de acceso como los getters.

16. ¿Cuál es el aporte de este proyecto?

El proyecto demuestra cómo un problema real puede transformarse en una solución informática estructurada.

No se trata solamente de hacer operaciones matemáticas. El programa integra:

Programación Orientada a Objetos

+

Java

+

Servlets

+

JSP

+

Maven

+

Tomcat

La combinación de estas tecnologías permite construir una aplicación web donde los datos ingresados por el usuario son procesados mediante objetos Java y posteriormente presentados mediante una interfaz web.

CONCLUSIONES

La actividad permitió aplicar de manera práctica los fundamentos de la Programación Orientada a Objetos. La creación de las clases `CompraProducto` y `CompraPasaje` permitió representar mediante objetos situaciones reales relacionadas con compras de productos y pasajes.

El encapsulamiento se aplicó mediante atributos privados y métodos públicos de acceso. Esto permite proteger la información interna de los objetos y controlar la forma en que otros componentes pueden consultar sus valores.

La división de los cálculos en métodos independientes permite desarrollar un código más organizado. Cada método tiene una responsabilidad específica, como calcular el importe, determinar el descuento, calcular el pago final y determinar los obsequios.

El uso de Servlets permite separar el procesamiento de las solicitudes de la interfaz gráfica. Los Servlets reciben los datos enviados por el usuario, validan la información, crean los objetos correspondientes y envían los resultados hacia las páginas JSP.

El uso conjunto de Java, JSP, Servlets, Maven y Tomcat permite comprender cómo los conocimientos de Programación Orientada a Objetos pueden aplicarse al desarrollo de una aplicación web completa.

RECOMENDACIONES

Se recomienda mantener una separación clara entre la lógica de negocio, el procesamiento de solicitudes y la presentación de información.

También es recomendable utilizar nombres de clases, atributos y métodos que describan claramente su función, como ocurre con `calcularImporteCompra()` y `calcularImporteDescuento()`.

Se debe continuar utilizando validaciones para evitar que datos incorrectos ingresados por el usuario ocasionen errores durante la ejecución.

Finalmente, es recomendable seguir practicando la creación de clases y objetos, ya que estos constituyen la base de la Programación Orientada a Objetos y permiten desarrollar aplicaciones más organizadas, reutilizables y fáciles de mantener.